Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Лабораторная работа №5

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2024

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 5
Листинг: 6
main.cu 6

Задание:

Реализуйте транспонирование матрицы размерностью N*K без использования разделяемой памяти, с разделяемой памятью без разрешения конфликта банков и с разрешением конфликта банков. Сравните время выполнения соответствующих ядер на GPU. Для всех трёх случаев определите эффективность использования разделяемой памяти с помощью метрик nvprof или ncu.

Цель: приобретение навыков использования разделяемой памяти

Ход работы:

Напишем программу, которая содержит транспонирование матрицы размерностью N*K:

1. без использования разделяемой памяти;
2. с разделяемой памятью без разрешения конфликта банков;
3. с разделяемой памятью с разрешением конфликта банков;

Сравним время выполнения соответствующих ядер на GPU (код программы представлен в листинге).

Программа была протестирована при различных размерах матрицы.

Результат работы программы (для замеров времени выполнения использовался Nvidia Visual Profiler (NVVP):

Name	Invocations	Avg. Duration	Regs	Static SMem
transposeNaive(int*, int*)	1	8,801 µs	10	0
transposeNaiveWithSharedMemory(int*, int*)	1	13,217 µs	13	4096
bankConflictResolution(int*, int*)	1	33,633 µs	18	4224

Рис. 1 – Результат работы программы при размерах матрицы 128x128.

Name	Invocations	Avg. Duration	Regs	Static SMem
transposeNaive(int*, int*)	1	3,1366 ms	10	0
transposeNaiveWithSharedMemory(int*, int*)	1	1,15158 ms	13	4096
bankConflictResolution(int*, int*)	1	613,393 µs	18	4224

Рис. 2 – Результат работы программы при размерах матрицы 1024x2048.

Как можно увидеть из результатов:

- Для матрицы размером 128x128:

bankConflictResolution: хоть и решает проблему конфликтов банков памяти, но имеет наибольшую сложность из-за двух циклов внутри ядра, что приводит к дополнительным накладным расходам. Поэтому данное ядро выполняется дольше всех.

transposeNaive: это ядро самое простое и не использует разделяемую память, поэтому оно наиболее эффективно для небольших матриц, таких как 128x128.

- Для матрицы размером 1024x2048:

bankConflictResolution: поскольку матрица стала гораздо больше, проблема конфликтов банков памяти стала более заметной. Использование **bankConflictResolution** с оптимизациями для избежания этих конфликтов привело к лучшей производительности по сравнению с другими ядрами.

transposeNaive: для больших матриц использование разделяемой памяти более выгодным, чем простой подход **transposeNaive**. Поэтому данное ядро будет самым медленным при выполнении для матрицы размером 1024x2048.

Вывод:

- Мы реализовали транспонирование матрицы размерностью N*K без использования разделяемой памяти, с разделяемой памятью без разрешения конфликта банков и с разрешением конфликта банков.
- Сравнили время выполнения соответствующих ядер на GPU для всех трёх случаев определите эффективность использования разделяемой памяти с помощью метрик nvprof (NVVP). При маленьких размерностях матрицы эффективней оказалось ядро без использования разделяемой памяти. При больших размерностях – с разделяемой памятью с разрешением конфликта банков.
- Приобрели навыки использования разделяемой памяти

Листинг:

main.cu

```
#include <stdio.h>

#define N 1024
#define K 2048
#define BLOCK_SIZE 32

__global__ void transposeNaive(int* in, int* out) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < N && y < K) {
        out[x * K + y] = in[y * N + x];
    }
}

__global__ void transposeNaiveWithSharedMemory(int* in, int* out) {
    __shared__ int cache[BLOCK_SIZE][BLOCK_SIZE];

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    int index_in = y * N + x;

    if (x < N && y < K) {
        cache[threadIdx.y][threadIdx.x] = in[index_in];
    }
    __syncthreads();

    x = blockIdx.y * blockDim.y + threadIdx.x;
    y = blockIdx.x * blockDim.x + threadIdx.y;

    int index_out = y * K + x;

    if (x < K && y < N) {
        out[index_out] = cache[threadIdx.x][threadIdx.y];
    }
}

__global__ void bankConflictResolution(int* in, int* out) {
    __shared__ int tile[BLOCK_SIZE][BLOCK_SIZE + 1];

    int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;

    for (int j = 0; j < BLOCK_SIZE; j += blockDim.y) {
        if (x < N && y + j < K) {
            int index_in = (y + j) * N + x;
            tile[threadIdx.y + j][threadIdx.x] = in[index_in];
        }
```

```
    }

    __syncthreads();

    x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
    y = blockIdx.x * BLOCK_SIZE + threadIdx.y;

    for (int j = 0; j < BLOCK_SIZE; j += blockDim.y) {
        if (x < K && y + j < N) {
            int index_out = (y + j) * K + x;
            out[index_out] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}

int main() {
    int* in, * out;
    cudaMallocManaged(&in, sizeof(int) * N * K);
    cudaMallocManaged(&out, sizeof(int) * N * K);

    for (int i = 0; i < N * K; i++) {
        in[i] = i;
    }

    dim3 grid((N + BLOCK_SIZE - 1) / BLOCK_SIZE, (K + BLOCK_SIZE - 1) /
BLOCK_SIZE);
    dim3 block(BLOCK_SIZE, BLOCK_SIZE);

    transposeNaive << <grid, block >> > (in, out);
    cudaDeviceSynchronize();

    transposeNaiveWithSharedMemory << <grid, block >> > (in, out);
    cudaDeviceSynchronize();

    bankConflictResolution << <grid, block >> > (in, out);
    cudaDeviceSynchronize();

    /*printf("Input matrix:\n");
    for (int i = 0; i < K; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", in[i * N + j]);
        }
        printf("\n");
    }

    printf("Output matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < K; j++) {
            printf("%d ", out[i * K + j]);
        }
        printf("\n");
```

```
    }*/

    printf("OK (%dx%d)", N, K);
    cudaFree(in);
    cudaFree(out);

    return 0;
}
```